extra metadata, as shown in listing 31.11. Your middleware is hooked into the endpoint routing system and benefits from everything it offers.

The example in listing 31.11 used a basic route template, `"/ping"`, but you can also use templates that contain route parameters, such as `"/ping/{count}"`, as you would with minimal APIs. The big difference is that you don't get the benefits of model binding that you get from minimal APIs, and it clearly takes more effort than using minimal APIs!

TIP For examples of how to access the route data from your middleware, as well as best-practice advice, see my blog entry titled "Accessing route values in endpoint middleware in ASP.NET Core 3.0" at http://mng.bz/4ZRj.

Converting existing middleware like `PingPongMiddleware` to work with endpoint routing can be useful when you have already implemented that middleware, but it's a lot of boilerplate to write if you want to create a new simple endpoint. In almost all cases you should use minimal API endpoints instead. But if you ever find yourself needing to reuse some existing middleware as an endpoint, now you know how!

In the next section we'll move away from the middleware pipeline and look at how to handle a common configuration requirement: using DI services to build a strongly typed `IOptions` objects.

31.2 Using DI with OptionsBuilder and IConfigureOptions

In this section I describe how to handle a common scenario: you want to use services registered in DI to configure `IOptions<T>` objects. There are